

language which communicates with hardware is called as low level language (assembly level)

language which does not communicate with hardware is called high level language (python, java)

php helps to build website the famous website is Wikipedia

using java we can develop application: - android, web, desktop, enterprise, cloud, testing tool

java is platform independency this major advantage over c, c++

platform means combination of processor and OS

flow chart c program or platform dependency

every assembler has there own proceesor

source code--->c complier---> assembler[binary code] ----> processor

platform independency

source code [in .java] ----->java complier[checks synatic][generate byte code] [coverte to .class]-----> JVM

java virtual machine[jvm is added on top of processor that help execute in any processor]-----> jvm can process in any process like window, ios

platform =os+ processor

there are two in execution

one is compliation , another is execution

1st we compile using javac file_name or class_name with extension

2nd in execution java only file_name will done by JVM

main is a method where jvm is look for main

variable and datatype

variable : registered area allocated on java memory / is piece of memory which contain a memory

four type of variable

instance

class

local

parameter

there are two type of datatype

primitive datatype : anything that start from lower case total 8 byte,short,long,int,char, double,float,boolean

Object references : anything that user define using Upper case like Class name, String, System

datatype :

2 steps to use variable

1) declaration --- Datatype and variblename

2) initialization ---variableName and reference/data/value

Datatype and variblename =reference/data/value

outside --meta space

inside -- stack

JVM

1. loads the byte code --> jvm help to load the bytecode

2. verifies the byte code --->valid or not the verifies whether the any security has been violating or not

3. READ The byte code stream line by line and execute the same

4. provide the runtime environment for execution

jvm-----class loader ----- byte code verfier----interpreter handle run time

jdk handles compile time which have java compiler or javac

runtime is handle by Jre which contain JVM

jre provide run time evironment

jvm is actually create run time environment

JRE contain JVM +set of library

JRE help to execute the code

JDK contain JRE

JRE is the minimum required to execute the java code(byte code)

JDK-- fully featured software development kit

JDK=JRE + development tools(compiler,debugger,java docs)

JRE= JVM +set of libraries

jre help to execute java code

jdk do programming + execute the code

all the component are platform dependent

only bytecode is the platform independent

memory

Stack -- the main purpose of stack is to execute main method.

inside stack Frame is created for main, println

local variable are store in Stack Main[frame] that will point to constant pool for variable

Ex:-

```
String team1 ="Mi"
```

the team1 is stored in the main [frame] the "mi" is stored in Constant pool

all the values is stored in heap memory

class related is store in metaspace like information of class

all the values are store in constant pool frame

methods

public static is -----access specifier

void is ----- return type

arrays

definition : group of variable of similar data type

syntax:

```
int i[];
```

```
int[] i;
```

```
int []i;
```

={}-----literal type without using new keyword

the output of every relation operater type is Boolean

method is only one which functionality of a class

methods:-

methods are the place where you put operation on data

definition :block of code / group of java statement which is used to perform operation on data(variable)

java methods must be d

parameter it is type of variable used inside the parentises to pass the value

use to pass the value

Functionalities/behaviour/purpose

advantage :-

types of method :- static and non Static

4 variable

class -----1)static 2)instance

class-----method----1)local 2)parameter

returnType=

<http://jenkov.com/>

java works on callbyvalue not ref

method overloading :- in a class 2 methods has same method_name but difference in parameter

ex : search(String foodName)--->search(String foodName,int Quantity)

static is used because it is used load/compile only once

class /template

is used to design the property and method is used defines once

class used to define the original copie

object :- to create multiple copies of class we used object

Indian rupee /bill

class :- is a design /template which contain properties(variable)and behaviour(methods).

static represent single copie

jit

javap class_name:- give the details what has been done in compiler

syntax

```
Bill var_name =new Bill();
```

```
className ref =new Constructor;
```

Constructor is special method

which is done by compiler

instance variable is always initialized with the Reference which is created in copies /objects

memory management

class information are stored in meta space

initially all the information is stored metaspace

what is stack is used in jvm

stack is used in jvm for execution of main method of the class

```
stack ---ensureopen()---write()---print (String x)-----println(),-----main ref[] -String  
,local,reference variable ,metadata like current line and new line information
```

new keyword is used to create copies, the variable in meta space are taken to instance memory which is in the heap memory

in instance variable

array we can not initialization directly

like using ref.array_name

to use the array and initial for the instance variable

for initial the array of the instance variable we have to use external variable(local variable)

example

we will create local variable array and then we will that local array to the instance variable array

=-----

Constructor

it is special type of a method declared with same name as class name

use case: initialize instance variable

type : default ,parameterized

helps in pass the value at time of object creation

this(); is used to do constructor chaining

to call constructor inside another constructor in the same class

constructor chaining can be done only the 1st statement

has a relationship

```
class HotelRunner {
```

```
    public static void main (String taj[]){
```

```
        Floor floor = new Floor;
```

```
Hotel hotel = new Hotel();  
    // has-A-relationship  
    String hotelName = "";  
    hotel.hotelName = hotelName ;
```

```
    hotel.floor = floor;
```

```
    Room room = new Room();
```

```
    floor.room = room;
```

```
    hotel.getHotelInfo();
```

what

why

type

where

how :Syntax

we create package for easy to access better increase readability and reuseability

package

two type

custom package: user defined

inbuilt package

java.lang is the default package that is available in java

there are 14 inbuilt package in the java

private variable is only accessed within that class only

private enable new concept encapsulation

to access that private variable we use getter(accessor) and setter(mutator)

this.application=application;// important if not we want create object every time we add the details

encapsulation: wrapping /binding the group of the data into a single unit/ block

private

class :5 rules : public class

default constructor

private.

getter and setter

implements

we can use parameterized constructor is not come under rules.

we use the encapsulated class as a parameters or a object reference/ custom type variable .

polymorphism is the extended part of the inheritance without inheritance there is no polymorphism

---class

1) variable 2)methods

variable -4type parameter, instance ,static ,local

methods -2type --static and instance

--objects

1)states

2)behaviour

static variable are shared among the different object

instance variable are can not shared among the different object

object class is the root class of the all class are called implicit inheritance.

inheritance: acquiring the properties and methods from the one class to another class

sharing a functionalities from main class to sub class.

using extends keyword inheritance can be achieved and inheritance is also known is a relation

advantages : code reusability

coupling

tight coupling :if change any code or implementation that effect the user that will called as tight coupling code or implementation.

which will not effect the will called as loose coupling

in industry they will always try to write loose coupling code

polymorphism is used only to achieve the loose coupling in the application.

interview question

String is magic class

object creation called String literals

```
String str = "";
```

in this once we create it will check whether the value is present the memory are not if it present it will error as it already defined

using new keyword

```
String str = new String();
```

in this type when it see the new keyword it will create an object it will not check in the memory wheather it present or not

== is always compare the address/ reference

if the value as a same "value" in

str

java is not a purely object oriented programing language because it consists of a primitive datatype which taken from the previous existing language

to overcome or to make class as purely object oriented class instead of primitive datatype use wrapper class which is build use primitive

ex: for int --> Integer class

long -->Long

boolean -->Boolean

with primitive so

wrapper class for all 8 primitive

```
System.out.println(id==id1); // it is comparing the reference /address of the variable
```

```
System.out.println(str.equals(str1)); //it is comparing the value of the variable
```

can we access the static member of the parent class in child using child className / refernce of the object

yes, we can access but it not a good partice it may violate rules in java

if parent has no default constructor then the child class also donot have default constructor

if we donot mention `@Override` also we can override the method

`@Override` it basically to define the complier about the override.

---garbage collection --memory leak --- daemon thread

polymorphism: object change its behaviour at given instance of time

2type

compile time polymorphism

by method overloading

runtime polymorphism

by method overriding

`scanner.close();`- if we not close after using the scanner it may aquire more memory

`System.gc()`

call to the method is decided on the runtime is called runtime polymorphism

type casting

```
Parent parent = new Child();  
parent.doBusiness()
```

it will call the parent method only but while printing it will check whether the same is overridden or not if it is overridden then it will use the overridden method that is in child class.

it will check which type of the object is created it will check whether the method is overridden or not

Lombok :api

it is external java library which is used to automate the writing repeated code like getter setter.

Lombok is a Java library that aims to reduce boilerplate code in Java applications by automatically generating common methods and functionalities at compile time. This is achieved through the use of annotations, which are processed by Lombok's annotation processor during the compilation phase

```
String str[] = {"", "", ""};
```

this type of array initialization is used when we know the value.

```
Patient patients[] = new Patient[2]// this type of array initialization is used when we not know the value.
```

we can initial the size of the arrays using constructor

```
public Patient (int size){  
    patients = new Patient[size];  
}
```

Enum : it is special type of the java class to store collections of constants

this is another way to store constants in the enum

```
public Enum Gender{  
    MALE , FEMALE;  
}
```

this is another way to store constants in the class

```
public static final String MALE = "MALE"
```

valueOf() : is used convert String to Enum

toString : is used convert Enum to String

2 type of inheritance

implicit inheritance: In the absence of any other explicit superclass, every class is implicitly a subclass of Object.

explicit inheritance

explicit inheritance : achieved by using extends keyword

3 type:

single level

multilevel

Hierarchy

abstraction :is achieve using interface and abstract

method which do not contain body is called abstract method it is only created in the interface

Concrete: which contain body of the method

by default for the abstract method is public

any methods in interface are always abstract. By this abstract keyword is optional

2 type of implementation to create abstract method

create using interface : in this we are not mention explicitly mention the

the class which implements from the interface is called implementation class

for interface there is no object or instance

We create abstract methods in interface.

And we use implements keyword next to the another class name **where this class helps to implements the methods helps to implements all the methods which is present in interface**. Where this class is called as implementation class.

We can extend and implement to a single class. But first we need to extend and then implement

Like :- class a extends b implements c {}

If we use abstract class then we need not to implement all the methods from interface. Then we should implement remaining abstract methods in child class.

We can create interface type of variable and implementation class type of constructor

Ex:- Interface ref = new implementation class();

Interfaces cannot have objects because they are considered to be incomplete blueprints

Interface:- is a component whose purpose is to provide abstract methods / behaviours helps to achieve abstraction.

Abstraction:-it's a process of hiding implementation and allowing the users to access only functionalities with the help of interface.

The outcome of abstraction is loose coupling. Loose coupling is achieved by interface.

Interface: keyword -> type

When 2 types are of same type we use is-a-relation

When 2 types are of different type we use has-a-relation

Inheritance Example in inbuilt :- Integer extends Number, Float, Double is a Number, where Number is Abstract class.

Classes takes fundamental nature from parent class

But interface provides methods

Class:- we use class when we know complete representation of an object.

AbstractClass : - we use this when we know partial representation of an object.

Interface :- we use this this when we don't know complete representation of an object

ExceptionHandling

A unwanted/unpredictable/abnormal event that occurred in middle of a code execution which was unpredictable which get triggered by JVM is known as Exception. To overcome this we use exception handling

Exception is class at compile time and object at runtime which extends Throwable.

This concept helps Java to be robust.

JVM will throw exception in runtime where all exceptions is a class . jvm throw this exception class as a object from java library to our code where problem occurs

Step 1 : a problem or exception is occurred.then its goes to JVM.

Step 2: JVM crete object or instance of Exception Class /

Step 3: JVM throws that created object to the program where problem occurred

Step 4: to handle that exception object we use try and catch().

```
Try{  
}  
}catch(Exception e){  
e. printStackTrace();  
}
```

e. printStackTrace():- helps to know about exception. what , where , when ,how

step by step and carefully analyze what happens if you don't handle an exception with try-catch in Java.

1. JVM looks up the call stack

- The JVM starts checking the method that caused the exception.
- If that method doesn't handle it, it moves up to the caller method, and so on.

2. Uncaught exception propagates to main()

- If no method handles it, the exception eventually reaches the main() method.

3. Program terminates abnormally

- JVM prints the exception type, message, and stack trace on the console.

Object is the parent most class where its sub class is Throwable where it has 2 subclasses Error and Exception.

Error:-it will be handled by JVM. When a infinite recursion occurs then we get runtime error StackOverFlowError.

Exception handled by us.

StackOverFlowError:-we get this error when a method is called again and again until it comes out of stack

OutOfMemoryError:-

There are 2 types of Exceptions :-CompileTimeException(Checked Exception) ,
RunTimeException(UnChecked Exception).

CompileTimeException: - doesn't extends RunTimeException. Example:- IOException, FileNotFoundException, ClassNotFoundException, SQLException. Will be handled at compileTime but occurred at runtime. And doesn't allow to run the program.

RunTimeException:- any extends this class comes under RunTimeException. Handled at run time and occurred at runtime. Allows to run the programz

If custom exception extends RunTimeException then its called Unchecked Exception.

If custom exception extends Exception then its called checked Exception.

Object Class

If we print object reference – it gives address like--com.xworkz.watchapp.watch.Watch@63947c6b(Fully qualified class Name through - String representation of the object)

Jvm invoke toString method to give address

mark Interface-which interface can not contain anything

11 methods in object

Equals, hash code, toString,wait(),wait(long time),wait(long time, int),notify(),notifyAll(),finalize(),getClass(),clone

If we override clone method we want to implement clonable

sop(object_ref)→

fully qualified class name with hexadecimal

this called string representation of an object

regularly used methods

equals()

hashCode()

toString()

when we try to print the reference of the Object so jvm will call the method of Objectclass method name called toString()

where it will return fully qualified class name with hexadecimal

for the user we should not represent the fully qualified class name with hexadecimal to the user we should display the state for the user.

so we are overriding the method toString to give the states for the user

toString are used in the encapsulated class

marker interface : the interface does not consists of abstract methods is know as marker interface

ex: cloneable , serialization

cloneable :

clone method

if we want clone the class then that Class must be implements the interface called Cloneable interface

when we compare two object reference like

book == book1 // fasle it compare the address/reference

book.equals(book1) // fasle if we compare two object if we call equals methods from the object class in that class also compare address only.

upcasting parent p = new child

downcasting Book book = (Book) o // ChildClass childref =(ChildClass)parentref

we cannot compare different Type of the refernce. or else we will get ClassCastException.

```
public boolean equals(Object obj) {  
    if (obj instanceof Book) {  
        Book book = (Book) obj; // down casting  
        if (this.bookId == book.bookId) {  
            return true;  
        }  
    }  
    return false;  
}
```

Upcasting and down casting

equals method comparing for value not reference

hashCode is just a system generated number it is native means which is got from the c++

String Class is an immutable class once object is create we cannot change if we reinitialize the value it will create object when we reinitialize the value

Objects Class : utility

static methods

ex: hash ,Nonnull

Collection Framework:

A collections framework is an architecture for representing and manipulating collections of objects. mainly consists of interfaces, implementation class, methods.

A collections framework is a unified architecture for representing and manipulating collections, enabling collections to be manipulated independently of implementation details.

framework : ready available

collection:

A collection is Interface where it is an object that represents a group of objects in order to overcome the drawback of array the collection framework come into exist.

collection is in java util package

collection is a interface in java

Array does not have inbuilt methods drawBack

collection have inbuilt methods

to overcome the draw back of array that why collection come

in collection we can add objects only

if we pass primitive datatype to the add() collection but in java it will convert(this conversion is know as autoboxing) primitive to Object

ex:

if int a = 20;

collection.add(a) a is converted to Integer type and added as objects

collection help to store homogenous and heterogenous

each and every concept in collection are built using data structure

for ArrayList is works on data structure is resizable or growable Array

collection (I) mainly contain 3 class

1)List (I) -- implementation class --ArrayList ,LinkedList, vector , Stack

2)Queue (I)

3)set (I)

list : list is used add the duplicate values and null values it give ordered collection

list preserve the order of insertion

set : set doesn't allow the duplicate values give random order

linkedlist will also act like queue

treeSet allow only type of the data in treeSet by default it will sort

used to sort

collection methods :

Boolean add(Object)

Boolean addAll(Collection)

Boolean remove(Object)

Boolean removeAll(Collection)

Boolean contain(Object)

Boolean containAll(Collection)

int size(Collection)

int hashCode()

Boolean isEmpty()

Boolean clear()

Boolean retainAll(Collection)

add method in collection

boolean add(Object ob)

JAVA List:

represent element in index manner.

<> Generics : this is used to store the same type

<Integer>

the default capacity of the array list is 10.

if we need to sort custom object then we need implements the comparable interface and @override the compareTo method

in this method the efficient way to compare is subtraction where if sub is -ve number then no need to swap if 0 then no need to swap if we get +ve number then we need to swap

Integer and String it already implements comparable

if we need to sort the custom id

@Override

```
public int compareTo(MobileInfo mobile) {  
    return this.mobileId - mobile.mobileId;  
}
```

if we need to sort the custom String type

then we have String.compareTo method which available in the String class

if we need to sort using single field use Comparable package and override the compareTo method , if we need to sort using more one field like Example : i want sorting using id and also app name so we can not override the 2 times in the same class so

id sorting is done by Comparable interface by Overriding the compareTo(Object o) methods

For other we need to create another Class AppNameComparator which implements Comparator and Overriding the compare(Object o1 ,Object o2)

ex:

```
public class AppNameComparator implements Comparator<AppInfo> {  
    @Override  
    public int compare(AppInfo app1, AppInfo app2) {  
        return app1.getAppName().compareTo(app2.getAppName());  
    }  
}
```

for Sorting we use Collections.sort(arrayListType, new AppNameComparator)

comparable interface compareTo can be override only once , it is known as default sorting.

Comparator interface compare can be override multiple times , it is known as custom sorting.

LinkedList: when every element is linked with both previous and next node is called LinkedeList
add()---linkLast(e)---

linked list is built on data structure called doubly linked list

linked list is grow by one at a time

for adding element to list LinkedList is best implementation because in arraylist array is immutable when we add it will add and create a new array copy so linked list is best implementation.

for fetch the element in the linked list it uses for loop

Set

set allow the custom duplicate if we do not use the equals and hashCode methods in the encapsulated class
if we use that it will not allow the duplicate

set allow implementation of the map

hashset uses hashmap

put method help to store as key and value

map use key value pair

HASH SET IMPLEMENTATION:

add("101") Boolean ----map.put("101",present) == null ----class HashMap -- key and value

Queue

queue will store the value as Object[] array

The remove() and poll() methods remove and return the head of the queue. Exactly which element is removed from the queue is a function of the queue's ordering policy, which differs from implementation to implementation. The remove() and poll() methods differ only in their behavior when the queue is empty: the remove() method throws an exception, while the poll() method returns null.

Queue implementations generally do not allow insertion of null elements, although some implementations, such as LinkedList, do not prohibit insertion of null. Even in the implementations that permit it, null should not be inserted into a Queue, as null is also used as a special return value by the poll method to indicate that the queue contains no elements.

peek vs poll ,element, offer vs add interview

to work on Priority Queue the add data of the Priority Queue must be comparable or else that class must be implements the comparable class

treeSet it will works ds of red black tree

in order to retrieve the data we have for each and iterator

concurrentModificationException this exception will come when do the remove or add operation while for each to overcome this we are using iterator it help remove or add while iterating

syntax

```
Iterator<Trainee> traineeIterator = trainees.iterator();  
while(traineeIterator.hasNext()) {  
    if(traineeIterator.next().getId() == 1)  
        traineeIterator.remove();  
}  
System.out.println(trainees);
```

where in this iterator have a method called hasNext()--return boolean and next()-- return object

hasNext

iterator will be in the -1 position iterator.hasNext return Boolean false if there are no element is present if element is present return true ,

Next() method return the object Like trainee

platform

3 types

Jse(java standard edition): standalone application

Jee (java enterprise edition): execute web application or cloud

Jme(java micro edition) : to execute ..and iord application

java 8

from java 1.8 java language is known as java is a functional language

java 1.8 ?

to bring concise in the programming by using functional programming like lambda function.

lambda

stream api

date api

method references

functional interface

static & default methods

From java 1.8

Functional interface

two ways to define

- 1) `@FunctionalInterface` using this annotation.
- 2) interface with one abstract method

if any interface which consists of only one abstract method is called functional interface.

lambda -> (symbol) is an anonymous function invoked without access specifier and return type by only -> symbol.

lambda function is help to give implementation for functional interface

functional interface syntax :-

```
interface Addition(){  
    int addingTwoNumber (int x , int y)  
}
```

lambda function syntax:

```
Addition add = (x, y)->{  
    sop(x + y);  
}
```

```
add.addingTwoNumber(5,10);
```

functional interface used to give reference to the lambda function

lambda function is used to give implementation (get the reference) to functional interface.

from 1.8 interface can contain both concrete methods and abstract methods but that concrete method must be static or default.

the main functional interface :

Supplier -- get

Consumer --accept

Predicate --test --Boolean this are used in filter scenario

Function --apply

Consumer: It is used when you want to take one input and do something with it —
but you don't return anything.

syntax:

```
Consumer<String> consumer1 = (message) ->{  
    System.out.println("this is Consumer interface");  
};  
consumer1.accept("hi");
```

biConsumer: same as consumer but it takes two arguments

DTO -- Design Patterns

view -- controller-- service --- repo

the class which contain the data member where the data member is transfer from one layer to another layer
that class name with endof or suffix

DTO: data transfer object it is design pattern used which is contain data member.

example: ClassNameDto

Stream Api:

special type of iterator which is to process the data from the collection in the function manner
function manner means using Functional Interface and lambda function.

Stream will work on operation

there mainly 2 operation :

intermediate

terminal: it will close the stream or create another collection

if stream return a stream then it is called as intermediate operation

if stream return other than stream it is called as terminal operation.

```
Stream<String> countryStream =countries.stream();  
countryStream = countryStream.filter(country-> country.startsWith("I"));  
List<String> filteredCountries = countryStream.collect(Collectors.toList());  
System.out.println(filteredCountries);
```

```
System.out.println("sorting based on the prime minister name");  
  
Comparator<CountryDto> comparator = (o1, o2) ->  
o1.getPrimeMinisterName().compareTo(o2.getPrimeMinisterName());  
countryDtos.stream().sorted(comparator).forEach(System.out::println);
```

distinct

```
map(EmployeeDto::getDepartment).distinct()
```

for using reversed() we can not use o1,o2 lambda expression

```
employeeDto.stream().sorted(Comparator.comparing(EmployeeDto::getSalary).reversed()).skip(1).findFirst()
```

to call stream

using .stream() method

using Stream.of(CountryDto)